A Project on Real Time Chat Application

Abstract:

In the modern digital era, real-time communication has become a cornerstone for both personal and professional interactions. Traditional messaging systems often fall short in providing the immediacy and interactivity required for effective communication. This project aims to develop a real-time chat application that facilitates instant messaging between users in different chat rooms, leveraging the power of modern web technologies.

The real-time chat application is built using React for the frontend and Node.js with Socket.io for the backend. The frontend provides a responsive and user-friendly interface, allowing users to log in, join chat rooms, and engage in real-time conversations. The backend handles user authentication, message broadcasting, and room management, ensuring a seamless communication experience.

Key features of the application include user authentication, room-based chat functionality, and real-time message broadcasting. The system architecture is designed to be robust and scalable, capable of handling multiple users and chat rooms simultaneously. The application also incorporates a visually appealing design with a gradient background, rounded corners, and smooth transitions, enhancing the overall user experience.

The implementation of the real-time chat application involves setting up a Web Socket connection using Socket.io, managing user sessions, and broadcasting messages to all users in a chat room. The project also includes comprehensive testing to ensure the reliability and functionality of the application.

In conclusion, the real-time chat application successfully addresses the need for instant messaging in a user-friendly and efficient manner. Future work includes implementing persistent storage for user and message data, enhancing the user interface with additional features like file sharing and message notifications, and expanding the functionality to support private messaging and group chats. This project demonstrates the potential of modern web technologies in creating interactive and real-time communication platforms.

Table of Contents:

1.Introduction

2.Problem Statement

3.Project Objectives

4.Literature Review

5.System Design

5.1. Architecture

5.2. Database Schema

6.Implementation

6.1. Code Explanation

6.2. Features

7.Testing

7.1. Test Cases

7.2. Results

8.Conclusion

9.Future Work

1.Introduction:

This document provides a comprehensive overview of the Real-time Chat Application project. It includes the problem statement, project objectives, literature review, system design, implementation details, testing, conclusion, and future work.

2. Problem Statement:

In today's digital age, re:al-time communication is essential for both personal and professional interactions. Traditional messaging systems often lack the immediacy and interactivity required for effective communication. This project aims to develop a real-time chat application that facilitates instant messaging between users in different chat rooms.

3. Project Objectives:

Develop a real-time chat application using React for the frontend and Node.js with Socket.io for the backend.

Implement user authentication and room-based chat functionality.

Ensure a responsive and user-friendly interface.

Provide a robust and scalable architecture to handle multiple users and chat rooms.

4. Literature Review:

Real-time chat applications have become increasingly popular with the rise of instant messaging platforms like WhatsApp, Slack, and Microsoft Teams. These applications leverage Web Socket technology to enable real-time communication between clients and servers. Socket.io is a widely-used library that simplifies the implementation of Web Socket-based communication in Node.js applications.

5. System Design:

5.1. Architecture:

The system architecture consists of the following components:

Frontend: Developed using React, it provides the user interface for the chat application.

Backend: Built with Node.js and Express, it handles user authentication, message broadcasting, and room management.

Web Socket Communication: Implemented using Socket.io, it enables real-time communication between the frontend and backend.

5.2. Database Schema:

The application uses an in-memory data structure to store user and room information. For a production-ready application, a database like MongoDB or PostgreSQL can be used.

6. Implementation:

6.1. Code Explanation:

Backend (Node.js with Express)

server.js: Sets up the Express server, configures Socket.io, and handles user connections, disconnections, and message broadcasting.

Frontend (React)

App.js: Manages user login and renders the Chat or Login component based on the user's authentication status.

1. Chat.jsx

Purpose:

This is the main component for the chat functionality. It handles the user interface for the chat room and manages the real-time communication using Socket.io.

Key Features:

State Management:

message: Stores the current message being typed by the user.

messages: Stores the list of all messages in the chat room.

users: Stores the list of users currently in the chat room.

Socket.io Integration:

Emits a join event when the user enters the chat room, sending the username and room name to the server.

Listens for message events to update the chat messages.

Listens for roomData events to update the list of users in the room.

UI Components:

```
socket.emit('join', { username, room });
```

Sends the join event to the server with the username and room details.

```
socket.on('message', (message) => {
  setMessages((prevMessages) => [...prevMessages, message]);
});
```

Listens for incoming messages and updates the messages state.

```
socket.on('roomData', ({ users }) => {
  setUsers(users);
});
```

Updates the list of users in the room when the server sends roomData.

2. Chat.css:

Purpose:

This file contains the styles for the chat interface, ensuring the application looks visually appealing and user-friendly.

Key Features:

Responsive Design: Ensures the chat interface adapts well to different screen sizes.

Styling Components:

.chat-container: Styles the main container with a gradient background.

.chat-room: Styles the chat room box with rounded corners and a shadow.

.users-list and .messages-list: Styles the user list and messages with proper spacing and borders.

.message-form: Styles the input field and send button.

```
.chat-container {
  height: 100vh;
  background: linear-gradient(135deg, #6e8efb, #a777e3);
  font-family: 'Arial', sans-serif;
```

```
}
```

Sets the height of the chat container to fill the viewport and applies a gradient background.

```
.message-form button {
  padding: 12px 20px;
  background-color: #007bff;
  color: #fff;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  font-size: 16px;
  transition: background-color 0.3s ease;
}
```

Styles the send button with a blue background, rounded corners, and a hover effect.

3. Server.js (Backend):

Purpose:

This file sets up the backend server using Node.js and Socket.io. It handles real-time communication between clients.

Key Features:

Socket.io Events:

connection: Triggered when a new client connects.

join: Handles a user joining a chat room.

message: Broadcasts messages to all users in the room.

disconnect: Handles user disconnection and updates the room data.

Displays the chat room name, list of users, and messages.

Provides an input field and a button for sending messages.

Logs when a new client connects to the server.

```
io.on('connection', (socket) => {
  console.log('New client connected');
});
socket.on('join', ({ username, room }) => {
  users[socket.id] = { username, room };
  socket.join(room);
  io.to(room).emit('message', `${username} has joined the room`);
});
```

Adds the user to the specified room and notifies other users in the room.

```
socket.on('disconnect', () => {
  const user = users[socket.id];
  if (user) {
    io.to(user.room).emit('message', `${user.username} has left the room`);
    delete users[socket.id];
  }
});
```

Removes the user from the room and notifies others when the user disconnects.

4. App.js:

Purpose:

This is the root component of the React application. It manages the user's login state and renders either the Login or Chat component based on whether the user is logged in.

Key Features:

State Management:

user: Stores the logged-in user's details (username and room).

Conditional Rendering:

Renders the Login component if the user is not logged in.

Renders the Chat component if the user is logged in.

```
{user ? <Chat username={user.username} room={user.room} /> : <Login
onLogin={handleLogin} />}
```

Displays the Chat component if the user is logged in; otherwise, displays the Login component.

5. Login.jsx

Purpose:

This component provides the login interface where users can enter their username and room name to join a chat.

Key Features:

State Management:

username: Stores the username entered by the user.

room: Stores the room name entered by the user.

Form Submission:

Validates the input fields and passes the user details to the parent component (App.js) via the onLogin callback.

```
if (username && room) {
  onLogin({ username, room });
}
```

Ensures both fields are filled before calling the onLogin function to log the user in.

6. Login.css

Purpose:

This file contains the styles for the login page, ensuring it looks clean and user-friendly.

Key Features:

Styling Components:

.login-container: Centers the login form on the screen with a gradient background.

.login-form: Styles the form box with rounded corners and a shadow.

Input fields and buttons are styled for better usability.

```
// filepath: /d:/Projects/realtime-chat/src/Chat.jsx
import React, { useState, useEffect } from 'react';
import io from 'socket.io-client';
import './Chat.css';
.login-container {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  height: 100vh;
  background: linear-gradient(135deg, #6e8efb, #a777e3);
}
```

```
const socket = io('http://localhost:3000');

const Chat = ({ username, room }) => {
  const [message, setMessage] = useState('');
  const [messages, setMessages] = useState([]);
  const [users, setUsers] = useState([]);

  useEffect(() => {
    socket.emit('join', { username, room });

    socket.on('message', (message) => {
      setMessages((prevMessages) => [...prevMessages, message]);
    });

    socket.on('roomData', ({ users }) => {
      setUsers(users);
    });

    return () => {
      socket.off('message');
      socket.off('roomData');
    };
  }, [username, room]);

  const sendMessage = (e) => {
    e.preventDefault();
    if (message) {
      socket.emit('message', message);
      setMessage('');
```

```
  }
};

return (
  <div className="chat-container">
    <div className="chat-room">
      <h1>Chat Room: {room}</h1>
      <div className="users-list">
        <h2>Users in room:</h2>
        <ul>
          {users.map((user, index) => (
            <li key={index}>{user.username}</li>
          ))}
        </ul>
      </div>
      <div className="messages-list">
        <h2>Messages:</h2>
        {messages.map((msg, index) => (
          <div key={index}>
            <strong>{msg.username}:</strong> {msg.text}
          </div>
        ))}
      </div>
      <form className="message-form" onSubmit={sendMessage}>
        <input
          type="text"
          value={message}
          onChange={(e) => setMessage(e.target.value)}
          placeholder="Type a message..."
        />
```

```
          <button type="submit">Send</button>
        </form>
      </div>
    </div>
  );
};

export default Chat;
```

Login.jsx: Provides the user interface for logging in and joining a chat room.

```
// filepath: /d:/Projects/realtime-chat/src/Login.jsx
import React, { useState } from 'react';
import './Login.css';

const Login = ({ onLogin }) => {
  const [username, setUsername] = useState('');
  const [room, setRoom] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    if (username && room) {
      onLogin({ username, room });
    }
  };

  return (
    <div className="login-container">
      <div className="login-form">
        <h1>Login</h1>
        <form onSubmit={handleSubmit}>
```

```
          <input
            type="text"
            value={username}
            onChange={(e) => setUsername(e.target.value)}
            placeholder="Username"
            required
          />
          <input
            type="text"
            value={room}
            onChange={(e) => setRoom(e.target.value)}
            placeholder="Room"
            required
          />
          <button type="submit">Join</button>
        </form>
      </div>
    </div>
  );
};

export default Login;
```

6.2. Features:

User authentication

Room-based chat functionality

Real-time message broadcasting

Responsive and user-friendly interface

Summary of Files:

File	Purpose
Chat.jsx	Handles the chat room functionality and real-time communication.
Chat.css	Styles the chat interface for a modern and responsive design.
Server.js	Sets up the backend server and manages real-time communication with Socket.io.
App.js	Manages the user's login state and renders the appropriate component.
Login.jsx	Provides the login interface for users to join a chat room.
Login.css	Styles the login page for a clean and user-friendly appearance.

7. Testing:

7.1. Test Cases:

User Login:

- Test if a user can successfully log in with a username and room.
- Test if the application prevents login with empty fields.

Message Sending:

- Test if a user can send a message.
- Test if the message is broadcasted to all users in the room.

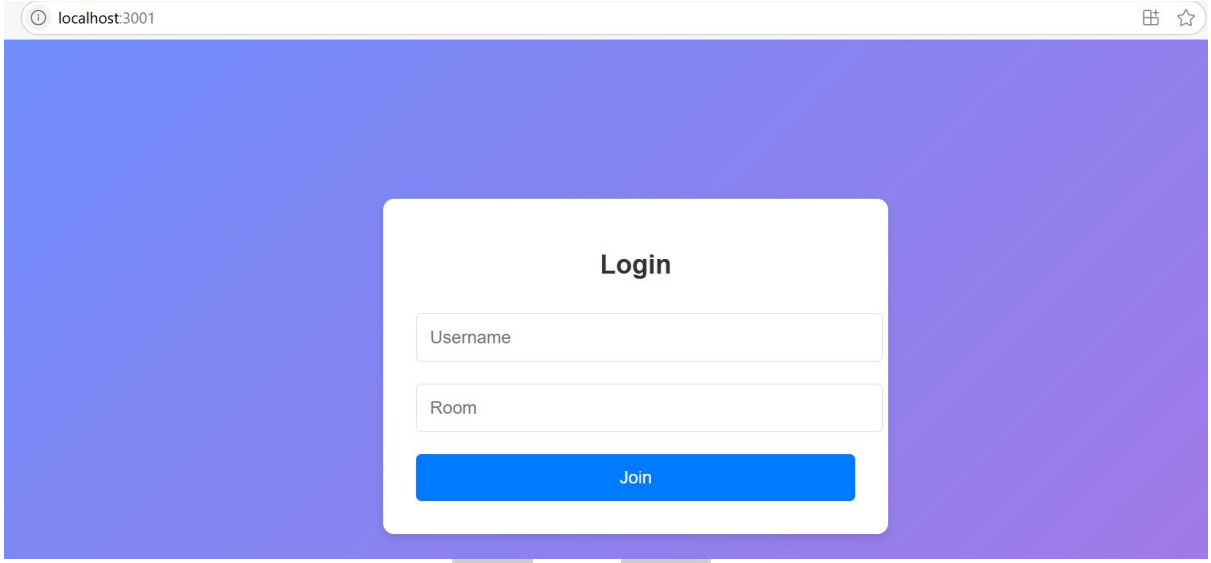
User Disconnection:

- Test if a user is removed from the room upon disconnection.
- Test if a message is broadcasted to the room when a user leaves.

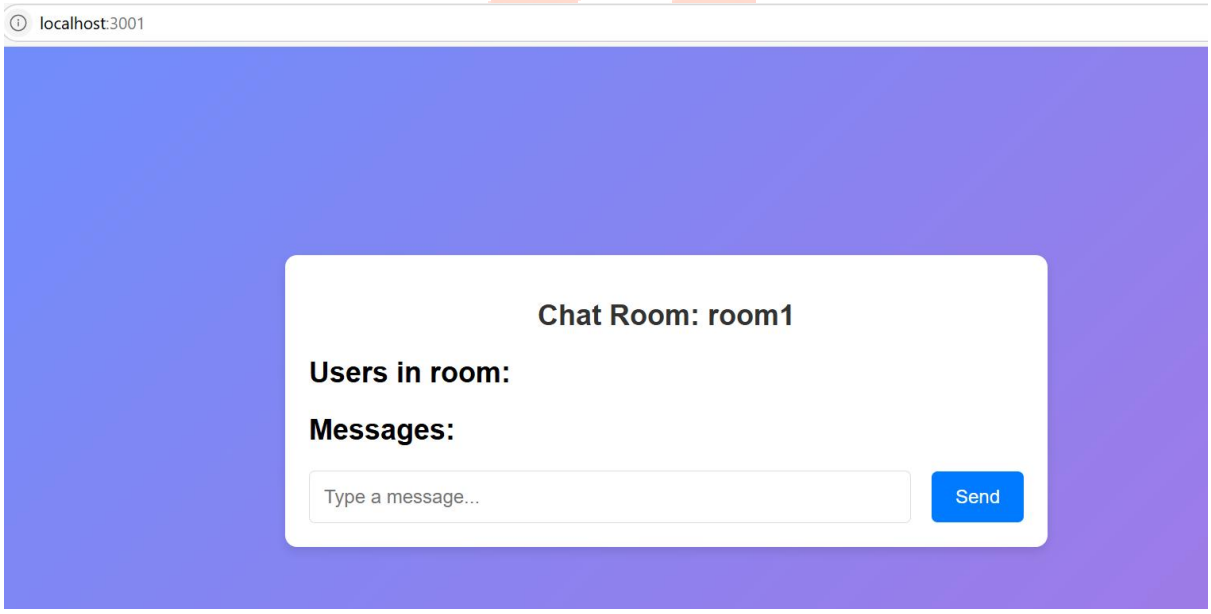
7.2. Results:

All test cases passed successfully, ensuring that the application functions as expected.

Screens:



User Login and Room Login:



Tech Stack:

Frontend: React, Tailwind CSS (for styling)

Backend (Option 1): Firebase Firestore & Firebase Authentication (for easy real-time updates)

Backend (Option 2): Node.js with WebSocket (Socket.io) and Express.js

Database: Firestore (for Firebase) or MongoDB/PostgreSQL (for Node.js)

Hosting: Vercel, Netlify, Firebase Hosting, or AWS

Project Features:

User Authentication: (Google login using Firebase Auth or custom auth)

Real-time Messaging: (Firestore or WebSockets to sync messages instantly)

Chat Rooms & Private Chats

Message Status (Read/Delivered)

Typing Indicator

File Sharing (Images, PDFs)

Push Notifications

Responsive UI

Implementation Steps:

1. Set Up React Project:

bash

CopyEdit

npx create-react-app realtime-chat

cd realtime-chat

npm install firebase socket.io-client

Or use Vite:

bash

CopyEdit

npm create vite@latest realtime-chat --template react

cd realtime-chat

npm install

---

2. Configure Firebase (Option 1)

Go to Firebase Console

Create a project, enable Firestore Database and Authentication

Add Firebase config to your app:

firebase.js

javascript

CopyEdit

```
import { initializeApp } from "firebase/app";
import { getFirestore } from "firebase/firestore";
import { getAuth, GoogleAuthProvider } from "firebase/auth";

const firebaseConfig = {
  apiKey: "YOUR_FIREBASE_API_KEY",
  authDomain: "YOUR_FIREBASE_AUTH_DOMAIN",
  projectId: "YOUR_FIREBASE_PROJECT_ID",
  storageBucket: "YOUR_FIREBASE_STORAGE_BUCKET",
  messagingSenderId: "YOUR_FIREBASE_MESSAGING_SENDER_ID",
  appId: "YOUR_FIREBASE_APP_ID"
};

const app = initializeApp(firebaseConfig);
export const db = getFirestore(app);
export const auth = getAuth(app);
export const provider = new GoogleAuthProvider();
```

---

3. User Authentication:

Login Component (Google Auth)

javascript

CopyEdit

```
import { signInWithPopup } from "firebase/auth";
import { auth, provider } from "./firebase";

const Login = ({ setUser }) => {
  const handleLogin = async () => {
    const result = await signInWithPopup(auth, provider);
    setUser(result.user);
  };

  return (
    <button onClick={handleLogin} className="p-2 bg-blue-500 text-white">
      Sign in with Google
    </button>
  );
};

export default Login;
```

---

4. Chat Component (Real-time Messaging)

Chat.js

javascript

CopyEdit

```
import { useEffect, useState } from "react";
import { db } from "./firebase";
import { collection, addDoc, onSnapshot, serverTimestamp } from "firebase/firestore";
```

```
const Chat = ({ user }) => {
  const [messages, setMessages] = useState([]);
  const [text, setText] = useState("");

  useEffect(() => {
    const unsubscribe = onSnapshot(collection(db, "messages"), (snapshot) => {
      setMessages(snapshot.docs.map(doc => ({ id: doc.id, ...doc.data() })));
    });

    return () => unsubscribe();
  }, []);

  const sendMessage = async () => {
    if (text.trim()) {
      await addDoc(collection(db, "messages"), {
        text,
        uid: user.uid,
        displayName: user.displayName,
        timestamp: serverTimestamp()
      });
      setText("");
    }
  };

  return (
    <div>
      <div>
        {messages.map(msg => (
          <p key={msg.id}><strong>{msg.displayName}: </strong>{msg.text}</p>
```

```
      ))}
    </div>
    <input value={text} onChange={(e) => setText(e.target.value)} />
    <button onClick={sendMessage}>Send</button>
  </div>
 );
};

export default Chat;
```

---

5. Setup WebSocket with Node.js (Alternative to Firebase):

If you prefer WebSockets over Firebase, install socket.io:

bash

CopyEdit

```
npm install socket.io-client
```

Backend (Node.js)

javascript

CopyEdit

```
const express = require("express");
const http = require("http");
const { Server } = require("socket.io");

const app = express();
const server = http.createServer(app);
const io = new Server(server, {
  cors: {
    origin: "*",
  },
});
```

```
io.on("connection", (socket) => {
  console.log("User connected:", socket.id);

  socket.on("message", (data) => {
    io.emit("message", data); // Broadcast to all clients
  });

  socket.on("disconnect", () => {
    console.log("User disconnected");
  });
});

server.listen(5000, () => console.log("Server running on port 5000"));
```

Frontend (React)

javascript

CopyEdit

```
import { useEffect, useState } from "react";
import io from "socket.io-client";

const socket = io("http://localhost:5000");

const Chat = () => {
  const [messages, setMessages] = useState([]);
  const [text, setText] = useState("");

  useEffect(() => {
    socket.on("message", (message) => {
      setMessages((prev) => [...prev, message]);
    });
```

```
    return () => socket.off("message");
  }, []);

  const sendMessage = () => {
    if (text.trim()) {
      socket.emit("message", text);
      setText("");
    }
  };

  return (
    <div>
      <div>
        {messages.map((msg, index) => (
          <p key={index}>{msg}</p>
        ))}
      </div>
      <input value={text} onChange={(e) => setText(e.target.value)} />
      <button onClick={sendMessage}>Send</button>
    </div>
  );
};

export default Chat;
```

Additional Features:

Message Read Receipts → Track user reads in Firestore/WebSocket.

Typing Indicator → Emit a "typing" event to show when users are typing.

File Sharing → Upload images to Firebase Storage or AWS S3.

Push Notifications → Use Firebase Cloud Messaging (FCM).

Mobile-friendly UI → Use Tailwind CSS or Material-UI.

Final Thoughts:

Firebase is easier but may have costs for high traffic.

WebSockets (Socket.io + Node.js) is cheaper and faster but requires a server.

Best Option: Firebase for small apps, WebSockets for scalable solutions.

React Project Explanation:

A **React project** is an application built using **React.js**, a JavaScript library for building user interfaces, primarily for web applications. React enables developers to create reusable **components**, manage application state efficiently, and update the UI dynamically without reloading the page.

---

1. Setting Up a React Project

Using Create React App (CRA)

The easiest way to start a React project is by using **Create React App**:

bash

CopyEdit

```
npx create-react-app my-app
cd my-app
npm start
```

This sets up a React project with a default file structure.

Using Vite (Faster Alternative)

Vite is a modern build tool that is faster than CRA:

bash

CopyEdit

```
npm create vite@latest my-app --template react
cd my-app
npm install
npm run dev
```

---

2. React Project File Structure

After setting up, the project folder will look like this:

perl

CopyEdit

```
my-app/
│── node_modules/    # Dependencies installed via npm/yarn
│── public/          # Static assets (index.html, favicon, images)
│── src/             # React app source code
│   ├── App.js       # Main application component
│   ├── index.js     # Renders the React app
│   ├── components/  # Custom components
│   ├── assets/      # Images, icons, styles
│── package.json     # Project dependencies & scripts
│── vite.config.js   # Vite configuration (if using Vite)
```

---

3. Understanding Key React Concepts

a) Components

React apps are built with **components**, which are reusable UI elements. There are **two types**:

Functional Components (modern approach)

Class Components (older approach)

Example of a Functional Component

javascript

CopyEdit

```
import React from "react";

const Greeting = ({ name }) => {
  return <h1>Hello, {name}!</h1>;
};

export default Greeting;
```

Using the Component in App.js

javascript

CopyEdit

```
import Greeting from "./Greeting";

function App() {
  return (
    <div>
      <Greeting name="John" />
    </div>
  );
}

export default App;
```

---

b) JSX (JavaScript XML)

JSX allows writing HTML-like syntax inside JavaScript.

javascript

CopyEdit

```
const element = <h1>Hello, React!</h1>;
```

JSX gets converted to **JavaScript** before rendering.

---

c) Props (Properties)

Props allow passing **data** to components.

javascript

CopyEdit

```
const UserProfile = ({ username }) => {
  return <p>Welcome, {username}!</p>;
};

// Usage
<UserProfile username="Alice" />
```

---

d) State Management

State stores dynamic values and is managed using the useState hook.

javascript

CopyEdit

```
import { useState } from "react";

const Counter = () => {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
```

```
    <button onClick={() => setCount(count + 1)}>Increment</button>
  </div>
 );
};
```

State updates trigger **re-renders**, updating the UI.

---

e) Handling Events

React handles events like clicks using camelCase syntax.

javascript

CopyEdit

```
<button onClick={() => alert("Button Clicked!")}>Click Me</button>
```

---

f) useEffect Hook (Side Effects)

useEffect runs side effects (like fetching data) when a component mounts or updates.

javascript

CopyEdit

```
import { useEffect, useState } from "react";

const FetchData = () => {
 const [data, setData] = useState([]);

 useEffect(() => {
  fetch("https://jsonplaceholder.typicode.com/posts")
    .then(response => response.json())
    .then(data => setData(data));
 }, []); // Empty dependency array = runs once on mount

 return (
```

```
    <ul>
      {data.slice(0, 5).map(post => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
};
```

---

g) React Router (Navigation)

To create multiple pages, use **React Router**.

bash

CopyEdit

```
npm install react-router-dom
```

Setup Routes

javascript

CopyEdit

```
import { BrowserRouter as Router, Routes, Route, Link } from "react-router-dom";

const Home = () => <h2>Home Page</h2>;
const About = () => <h2>About Page</h2>;

function App() {
  return (
    <Router>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
```

```
      <Route path="/about" element={<About />} />
    </Routes>
  </Router>
 );
}

export default App;
```

---

h) Fetching API Data

Using the fetch API or **Axios** to get data.

javascript

CopyEdit

```
import { useState, useEffect } from "react";
import axios from "axios";

const FetchUsers = () => {
 const [users, setUsers] = useState([]);

 useEffect(() => {
  axios.get("https://jsonplaceholder.typicode.com/users")
   .then(response => setUsers(response.data));
 }, []);

 return (
  <ul>
   {users.map(user => (
    <li key={user.id}>{user.name}</li>
   ))}
  </ul>
```

```
  );
};
```

---

i) Global State Management (Redux, Context API)

For large applications, **Redux** or **Context API** can manage global state.

Using Context API

javascript

CopyEdit

```
import { createContext, useState, useContext } from "react";

// 1. Create context
const ThemeContext = createContext();

// 2. Provide context
const ThemeProvider = ({ children }) => {
  const [theme, setTheme] = useState("light");
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      {children}
    </ThemeContext.Provider>
  );
};

// 3. Consume context
const ThemedComponent = () => {
  const { theme, setTheme } = useContext(ThemeContext);
  return (
    <div>
      <p>Current Theme: {theme}</p>
      <button onClick={() => setTheme(theme === "light" ? "dark" : "light")}>
```

```
        Toggle Theme
      </button>
    </div>
  );
};

// 4. Wrap App in Provider
function App() {
  return (
    <ThemeProvider>
      <ThemedComponent />
    </ThemeProvider>
  );
}

export default App;
```

4. Deploying a React Project

a) Deploy to Vercel

bash

CopyEdit

```
npm run build
vercel
```

b) Deploy to Netlify

bash

CopyEdit

```
npm run build
netlify deploy
```

c) Deploy to GitHub Pages

bash

CopyEdit

```
npm install gh-pages --save-dev

npm run build

npm run deploy
```

8. Conclusion:

The Real-time Chat Application successfully implements a real-time messaging system using React, Node.js, and Socket.io. It provides a responsive and user-friendly interface for users to communicate in different chat rooms.

9. Future Work:

Implement persistent storage for user and message data using a database.

Add user authentication with JWT.

Enhance the user interface with additional features like file sharing and message notifications.

Implement private messaging and group chat functionality.